

Virtual Wallet

Full-Stack Financial Application -- Technical Reference

Spring Boot 3

Java 17

Vue 3

TypeScript

MariaDB

Docker

Generated May 30, 2026

1. Project Overview 4

2. Getting Started 6

3. Authentication & Security 8

4. Features 10

5. Backend API Reference 13

6. Data Models 15

7. Frontend Architecture 17

8. Deployment 20

1. Project Overview

Virtual Wallet is a full-stack digital banking application that lets users register, verify their identity via email, manage multi-currency wallets (EUR, USD, GBP), link debit/credit cards, top up their wallets, and transfer funds between users with automatic real-time currency conversion. An admin panel allows staff to monitor all users and transactions across the system.

The backend is a Spring Boot 3 REST API (Java 17) backed by MariaDB, secured with stateless JWT authentication. The frontend is a Vue 3 + TypeScript SPA using Pinia for state management, Axios for HTTP, and Vue Router 4 for navigation. External currency conversion rates are fetched from ExchangeRate-API v6 in real time on each cross-currency transfer.

System Architecture

```

+-----+
|          BROWSER (Vue 3 + TypeScript)          |
| LoginPage RegisterPage DashboardPage WalletsPage |
| CardsPage TransactionsPage TransferPage Profile |
| Admin: UsersPage Admin: TransactionsPage         |
+-----+
|          | HTTPS /api/v0/*                      |
+-----+
|          SPRING BOOT BACKEND :8080              |
| AuthController WalletController CardController |
| TransactionController TransferController        |
| UserController                                  |
| JWT Filter -> SecurityFilterChain -> Service Layer |
+-----+
|          | JPA / Hibernate                      |          | RestClient
+-----+-----+-----+-----+-----+-----+
| MariaDB 11 :3306 | | ExchangeRate-API v6 |
| virtual_wallet DB | | Real-time FX rates |
+-----+-----+-----+-----+-----+

```

Technology Stack

Layer	Technology
Backend framework	Spring Boot 3.x (Java 17)
Database	MariaDB 11 (via Docker)
ORM	Spring Data JPA / Hibernate
Security	Spring Security + JWT (HS256, 24 h expiry)
Email	Spring Mail (Gmail SMTP)
Currency conversion	ExchangeRate-API v6
Frontend framework	Vue 3 (Composition API) + Vite
Language	TypeScript
State management	Pinia with persisted state plugin

HTTP client	Axios with request/response interceptors
Routing	Vue Router 4
UI icons	Lucide-vue-next
Dialogs	SweetAlert2 (custom themed)
Containerisation	Docker + Docker Compose

2. Getting Started

Prerequisites

- * Java 17 JDK
- * Docker Desktop (for MariaDB)
- * Node.js 20+ and npm
- * Git
- * An ExchangeRate-API key (free tier at exchangerate-api.com)
- * A Gmail account for SMTP (or any SMTP provider)

Backend Setup

1. Clone the repository and navigate to the backend directory:

```
git clone <repo-url>
cd virtual-wallet/backend
```

2. Create the secrets file (not committed to source control):

```
# src/main/resources/application-secrets.properties
exchange.rate.api.key=YOUR_EXCHANGERATE_API_KEY
spring.mail.username=your@gmail.com
spring.mail.password=your_app_password
```

3. Start the MariaDB database container:

```
docker-compose up -d
```

4. Build and run the backend:

```
./gradlew build
./gradlew bootRun
# API available at http://localhost:8080/api/v0
```

Frontend Setup

```
cd ../frontend
npm install
npm run dev
# Dev server at http://localhost:5173
```

Seeded Users (Development)

The DataSeeder configuration creates the following accounts on first startup:

Role	Username	Email	Password	Status
ADMIN	admin	admin@example.com	Admin1234!	Verified
USER	johndoe	john@example.com	John1234!	Verified
USER	janedoe	jane@example.com	Jane1234!	Verified

Running Tests

```
# All tests
./gradlew test

# Single test class
./gradlew test --tests example.backend.services.AuthServiceTests

# Verbose output
./gradlew test --info
```

Test Database

All tests use an H2 in-memory database configured via `@ExtendWith(MockitoExtension.class)`. Service tests inject mocked repositories so no running MariaDB instance is required.

3. Authentication & Security

JWT Token Flow

Authentication is stateless. The backend issues a signed JWT on login; every subsequent request must include it in the Authorization header. The server never stores session state.

Client	Backend	DB
-- POST /auth/login ---->		
{email, password}	-- findByEmail() ---->	
	<--- User -----	
	BCrypt.matches()	
	JWT.generate()	
<-- {token, user} -----		
-- GET /api/v0/* ----->		
Authorization:	JwtAuthFilter	
Bearer <token>	validates HS256	
	sets SecurityContext	
<-- 200 OK / 403 -----		

JWT Token Details

Algorithm: HMAC-SHA256 (HS256) | Expiry: 24 hours | Claims: sub (username), roles (array) | Token is also accepted from an HTTP cookie named 'jwt' as a fallback. The JwtAuthenticationFilter extends OncePerRequestFilter and runs on every request.

Email Verification (OTP)

New users must verify their email address before accessing protected features. On registration the backend generates a 6-digit code (ThreadLocalRandom), stores it with a 15-minute expiry, and sends it via Gmail SMTP. The user submits the code to POST /api/v0/auth/verify. On success the account is marked verified and a base EUR wallet is automatically created.

```

Register --> sendVerificationEmail() --> User submits code
          (6-digit OTP, 15 min expiry)    (POST /auth/verify)
                                     |
                                     isVerified = true
                                     createBaseEURWallet()

```

@RequiresVerifiedAccount Annotation

Sensitive operations (wallet creation, top-up, card management, transfers) are protected by the custom @RequiresVerifiedAccount annotation backed by a Spring AOP aspect. Before any annotated service method executes, the aspect retrieves the authenticated user, checks isVerified and isBlocked, and throws AccountNotVerifiedException

(HTTP 403) or `UserBlockedException` (HTTP 403) accordingly.

```
@RequiresVerifiedAccount // triggers AOP check before execution
public Wallet createWallet(Wallet wallet) { ... }

// Aspect logic (RequiresVerifiedAccountAspect.java):
@Before("@annotation(RequiresVerifiedAccount)")
public void check() {
    User user = authUtils.getAuthenticatedUser();
    if (!user.isVerified()) throw new AccountNotVerifiedException();
    if (user.isBlocked())   throw new UserBlockedException();
}
```

Roles & Permissions

Role	Capabilities
ROLE_USER	Login, register, verify, view own profile, manage own wallets/cards, make transfers, view own transactions, search
ROLE_ADMIN	All USER capabilities plus: list all users, get any user by ID, create users, block/unblock users, view all transactions, get transactions by any user, view any card by ID

Password Encoding

All passwords are encoded with BCrypt before storage (`BCryptPasswordEncoder` bean). Plain-text passwords are never persisted. Login validation uses `BCrypt.matches(raw, encoded)`. Admin-created users use a pre-set BCrypt-encoded password.

4. Core Features

Multi-Currency Wallets

Each verified user can hold up to three wallets -- one per supported currency (EUR, USD, GBP). A base EUR wallet is created automatically on first verification. Additional wallets can be named (max 20 characters) and are created immediately. Deletion requires a zero balance and at least one remaining wallet.

Rule	Detail
Currencies supported	EUR (EUR), USD (\$), GBP (GBP)
Unique constraint	One wallet per user per currency (DB-enforced)
Balance type	BigDecimal with precision 19, scale 2
Currency immutable	Cannot change currency after wallet creation
Deletion guard	Balance must be 0.00; user must retain ≥ 1 wallet
Auto-wallet	EUR wallet created automatically after email verification

Card Tokenisation

Users link debit/credit cards to their account. The full card number is never stored. The PaymentService tokenises each card on addition: it generates a unique tok_<UUID> token and a SHA-256 fingerprint of (cardNumber | brand) for deduplication. Only the token, fingerprint, last 4 digits, brand, and expiry are persisted.

Brand	Number Prefix	Length
VISA	Starts with 4	13 or 16 digits
MASTERCARD	51-55 or 22-27	16 digits
AMEX	34 or 37	15 digits
DISCOVER	6011 or 65	16 digits

Duplicate prevention: if a user tries to add the same physical card twice (same fingerprint), a DuplicateException (HTTP 409) is thrown. Expired cards are rejected at creation time.

Wallet Top-Up

A user may top up any of their own wallets using a linked card. The endpoint POST /api/v0/wallets/top-up receives the wallet ID, card ID, and amount. The PaymentService.charge() simulates a card charge (no live gateway), the wallet balance is incremented, and a TOP_UP transaction record is persisted with status SUCCESSFUL.

```
// TopUpRequest DTO
{
  "wallet_id": 1,
  "card_id": 2,
  "amount": 100.00
}
```

User-to-User Transfers

Transfers move funds between two verified users. The sender specifies their source wallet, the recipient's username, and the amount. If the currencies differ, the `ConversionService` fetches a live rate from `ExchangeRate-API v6` and applies it automatically. The recipient receives funds in their matching-currency wallet, or their first available wallet if no match exists.

```
TransferReq {fromWalletId, toUsername, amount}
  |
  v
1. Load source wallet (pessimistic lock)
2. Verify authenticated user owns source wallet
3. Load recipient by username -- must be verified
4. Find recipient wallet (prefer same currency -> fallback first)
5. Validate: different wallets, positive amount, sufficient balance
6. Convert amount if currencies differ (ExchangeRate-API)
7. Debit sender:  fromWallet.balance -= amount
8. Credit receiver: toWallet.balance += convertedAmount
9. Persist TRANSFER_OUT tx on source wallet
10. Persist TRANSFER_IN tx on dest wallet
```

Concurrency Safety

Wallets involved in a transfer are loaded with a pessimistic write lock (`SELECT ... FOR UPDATE` via `WalletRepository.findByIdForUpdate`). This prevents race conditions when two concurrent transfers attempt to debit the same wallet simultaneously.

Transaction History & Filtering

Transaction Type	Description
TOP_UP	Funds loaded from a linked card
WITHDRAWAL	Funds withdrawn (reserved for future use)
TRANSFER_OUT	Funds sent to another user (negative amount)
TRANSFER_IN	Funds received from another user (positive amount)
FEE	Service fee (reserved for future use)
REFUND	Refund credit (reserved for future use)

Status	Meaning
PENDING	Created but not yet settled (default on creation)
SUCCESSFUL	Operation completed successfully
FAILED	Operation failed (e.g., insufficient funds at gateway level)

Available filter parameters on `GET /api/v0/transactions`:

Parameter	Type	Description
type	Enum	Filter by transaction type (TOP_UP, TRANSFER_IN, ...)
status	Enum	Filter by status (PENDING, SUCCESSFUL, FAILED)

from	ISO DateTime	Include only transactions after this timestamp
to	ISO DateTime	Include only transactions before this timestamp
transactionId	Long	Exact transaction ID lookup
targetUserId	Long	Admin: filter by wallet owner's user ID
page	Integer	Zero-based page number (Spring Pageable)
size	Integer	Page size (default 20)

5. Backend API Reference

Base URL: `http://localhost:8080/api/v0` | All requests/responses use JSON (snake_case field names).

Auth Endpoints (/api/v0/auth)

Method	Path	Auth	Description
POST	/auth/login	None	Login with email + password. Returns JWT token and user profile.
POST	/auth/register	None	Register a new account. Sends a 6-digit OTP to the given email.
POST	/auth/verify	None	Submit the OTP code to verify the account and unlock features.
GET	/auth/me	JWT	Return the authenticated user's own profile (PrivateUserDto).

Wallet Endpoints (/api/v0/wallets)

Method	Path	Auth	Description
GET	/wallets	JWT	List all wallets belonging to the authenticated user.
GET	/wallets/{id}	JWT	Get a specific wallet by ID (must be owner).
POST	/wallets	JWT+Verified	Create a new wallet. Body: {name?, currency?}. EUR default.
DELETE	/wallets/{id}	JWT+Verified	Delete wallet. Fails if balance > 0 or it is the last wallet.
POST	/wallets/top-up	JWT+Verified	Top up wallet from a card. Body: {wallet_id, card_id, amount}.

Card Endpoints (/api/v0/cards)

Method	Path	Auth	Description
GET	/cards	JWT	List all cards for the authenticated user.
GET	/cards/{cardId}	JWT+Admin	Get any card by ID (admin only).
GET	/cards/users/{userId}	JWT+Admin	Get all cards belonging to a specific user (admin).
POST	/cards	JWT+Verified	Add a new card. Tokenised; full number never stored.
DELETE	/cards/{cardId}	JWT	Remove a card (card owner only).

Transaction Endpoints (/api/v0/transactions)

Method	Path	Auth	Description
GET	/transactions	JWT	Paginated, filterable list of the current user's transactions.
GET	/transactions/{id}	JWT	Get one of the current user's transactions by ID.
GET	/transactions/all	JWT+Admin	Paginated, filterable list of ALL transactions (admin).
GET	/transactions/admin/{id}	JWT+Admin	Get any transaction by ID regardless of owner (admin).

Transfer Endpoint (/api/v0/transfers)

Method	Path	Auth	Request Body
POST	/transfers	JWT+Verified	{from_wallet_id: Long, to_username: String, amount: Decimal}

Both sender and recipient must be verified. Insufficient funds throw `ImpossibleOperationException` (HTTP 400). Currency conversion is applied automatically.

User Endpoints (/api/v0/users)

Method	Path	Auth	Description
GET	/users	JWT+Admin	Paginated list of all users.
GET	/users/{userId}	JWT+Admin	Get any user's full profile by ID.
GET	/users/search	JWT	Search users by username/name. Query param: q=<string>.
POST	/users	JWT+Admin	Admin creates a new pre-verified user.
PUT	/users	JWT	Update own profile. Fields: email, phone_number, photo_url,
DELETE	/users	JWT	Delete own account (cascades to wallets and cards).

Error Response Format

All error responses follow a consistent structure:

```
{
  "status": 403,
  "message": "Account is not verified",
  "uri": "/api/v0/wallets",
  "errors": { "field": "validation message" }
}
```

Exception	HTTP Status	Scenario
<code>AuthorizationException</code>	403	User does not own the requested resource
<code>AccountNotVerifiedException</code>	403	Account requires email verification
<code>UserBlockedException</code>	403	User account has been blocked by admin
<code>DuplicateException</code>	409	Unique constraint violated (e.g. same card twice)
<code>EntityNotFoundException</code>	404	Requested resource does not exist
<code>ImpossibleOperationException</code>	400	Invalid operation (negative balance, same-wallet transfer)
<code>InvalidCardException</code>	400	Card number fails brand validation
<code>CardExpiredException</code>	400	Card expiry date is in the past
<code>ExchangeRateApiException</code>	503	ExchangeRate-API is unreachable

6. Data Models

All entities are JPA-mapped to a MariaDB 11 schema. Spring Data JPA handles DDL via `spring.jpa.hibernate.ddl-auto=update`. The naming strategy converts camelCase Java fields to snake_case column names automatically.

User

Field	Type	Constraints	Notes
id	Long	PK, auto-increment	
firstName	String	max 50	
lastName	String	max 50	
username	String	unique, not null	Used for login and search
password	String	not null	BCrypt-hashed; never plain-text
email	String	unique, not null, @Email	
phoneNumber	String	unique, E.164 format	+ [1-9] d {7,14}
photoUrl	String	optional, must start https://	
verificationCode	String	nullable	6-digit OTP
verificationCodeExpiresAt	LocalDateTime	nullable	15 min initial, 5 min resend
createdAt	LocalDateTime	@CreationTimestamp	Auto-set on insert
isBlocked	boolean	default false	Admin-controlled
isVerified	boolean	default false	Set to true after OTP verify
cards	Set<Card>	OneToMany, cascade ALL	Deleted with user
wallets	Set<Wallet>	OneToMany, cascade ALL	Deleted with user
roles	Set<Role>	ManyToMany	ROLE_USER ROLE_ADMIN

Wallet

Field	Type	Constraints	Notes
id	Long	PK, auto-increment	
owner	User	ManyToOne, not null	FK: user.id
name	String	max 20, nullable	Optional user-given label
balance	BigDecimal	precision 19, scale 2, default 0	Never negative
currency	Currency	Enum: EUR / USD / GBP, not null	Immutable after creation
transactions	Set<Transaction>	OneToMany	All transactions on this wallet

Unique Constraint

The combination of (owner, currency) has a database-level unique constraint. Attempting to create a second EUR wallet for the same user throws a `DuplicateException` (HTTP 409).

Card

Field	Type	Constraints	Notes
id	Long	PK, auto-increment	
token	String	unique, not null	tok_<UUID> -- payment gateway reference
fingerprint	String	unique per user	SHA-256(cardNumber brand)
cardBrand	CardBrand	Enum, not null	VISA / MASTERCARD / AMEX / DISCOVER
last4	String	not null	Last 4 digits for display
expirationMonth	int	not null	
expirationYear	int	not null	
cardHolder	User	ManyToOne, not null	FK: user.id

Transaction

Field	Type	Constraints	Notes
id	Long	PK, auto-increment	
amount	BigDecimal	precision 19, scale 2	Negative for debits
currency	Currency	Enum: EUR / USD / GBP	Currency of the amount
type	TransactionType	Enum, not null	See type table above
status	TransactionStatus	Enum, default PENDING	PENDING / SUCCESSFUL / FAILED
timestamp	LocalDateTime	@CreationTimestamp	Auto-set on insert
wallet	Wallet	ManyToOne, not null	Primary wallet involved
counterpartyWallet	Wallet	ManyToOne, nullable	Other wallet (transfers only)

Role

Field	Type	Notes
id	Long	PK, auto-increment
name	ERole	ROLE_USER or ROLE_ADMIN (Enum)

Entity Relationship Summary

```

User 1 ---< * Wallet      (user owns wallets)
User 1 ---< * Card        (user owns cards)
User * >---< * Role       (user has roles, ManyToMany)
Wallet 1 ---< * Transaction (wallet has transactions)
Transaction >--- Wallet   (counterpartywallet, nullable)

```

7. Frontend Architecture

Technology Stack

Package	Purpose
vue@3	Composition API, reactivity, template compiler
vite	Build tool and dev server (HMR)
typescript	Static typing throughout
pinia	Centralised state management
pinia-plugin-persistedstate	Persists auth store to localStorage
vue-router@4	Client-side routing with navigation guards
axios	HTTP client with interceptors
lucide-vue-next	Icon set (consistent throughout UI)
sweetalert2	Confirmation and notification dialogs (custom-themed)

Directory Structure

```

frontend/src/
  api/           # Axios modules per domain (auth, wallets, cards, ...)
  assets/        # base.css (design tokens) + main.css (global components)
  components/    # AppPagination.vue
  composables/   # Business logic hooks (useWallets, useCards, useTransfer, ...)
  layouts/       # DashboardLayout.vue (sidebar + main area)
  pages/         # Full-page route components
  router/        # index.ts (routes + beforeEach guard)
  stores/        # auth.ts + wallet.ts (Pinia)
  types/         # index.ts (all TypeScript interfaces / DTOs)

```

Pages & Routes

Route	Component	Guard	Description
/login	LoginPage	Guest only	Email + password login
/register	RegisterPage	Guest only	Registration + password strength check
/verify	VerifyPage	Guest only	OTP submission + auto-redirect
/	DashboardPage	Auth	Balance overview, recent transactions
/wallets	WalletsPage	Auth	Create, top-up, delete wallets
/cards	CardsPage	Auth	Add / remove cards with live preview
/transactions	TransactionsPage	Auth	Filtered, paginated transaction history
/transfer	TransferPage	Auth	User search + multi-step transfer form
/profile	ProfilePage	Auth	View own profile and verification status

/admin/users	Admin/UsersPage	Auth + Admin	Search and list all users
/admin/transactions	Admin/TransactionsPage	Auth + Admin	Full transaction ledger

Pinia Stores

auth store (stores/auth.ts) -- persisted to localStorage

State / Getter / Action	Type	Description
token	string null	JWT from login response
user	PrivateUserDto null	Current user's profile
emailForVerification	string null	Email used during verify step
isAuthenticated	computed bool	True when token is non-null
isAdmin	computed bool	True when user has ROLE_ADMIN
isVerified	computed bool	True when user.is_verified === true
login(creds)	action	Call authApi.login, store token + user
logout()	action	Clear token and user from state + localStorage
fetchMe()	action	Refresh user profile from GET /auth/me

wallet store (stores/wallet.ts) -- 30-second client-side cache

State / Action	Description
wallets: PrivateWalletDto[]	Array of the authenticated user's wallets
fetchWallets({ force })	Fetch wallets; skip if cache < 30 s old unless force=true
createWallet(payload)	POST /wallets, append result to state
deleteWallet(id)	DELETE /wallets/:id, remove from state
topUp(payload)	POST /wallets/top-up, then force-refetch wallets

API Client (api/client.ts)

A configured Axios instance is shared across all API modules. A request interceptor automatically injects Authorization: Bearer <token> from the auth store. A response interceptor catches HTTP 401 and calls authStore.logout() + router.push('/login') to handle token expiry transparently.

```
// Request interceptor
client.interceptors.request.use(config => {
  const token = authStore.token;
  if (token) config.headers.Authorization = `Bearer ${token}`;
  return config;
});

// Response interceptor
client.interceptors.response.use(null, error => {
  if (error.response?.status === 401) {
    authStore.logout();
    router.push('/login');
  }
});
```

Key Composables

Composable	Key Responsibilities
useWallets	Wallet CRUD modals, top-up modal, currency symbols, quick-add amounts
useCards	Card CRUD, brand detection (VISA/MC/AMEX), live card preview, expiry validation
useTransfer	Recipient search (300 ms debounce), wallet selection, transfer submission
useTransactions	Filter state (type/status/dates), 30-second page cache, smart pagination
useTransactionDetail	Lazy-loads full transaction on row expand; caches per ID
useDashboard	Parallel load: wallets + recent 5 transactions + card count
useFormatters	formatCurrency (Intl), formatDate, txLabel, statusLabel, amountClass
useDialog	SweetAlert2 wrapper: confirm(), error(), success() with custom theme

Design System

All CSS custom properties are defined in `assets/base.css`. Global component classes (buttons, inputs, modals, spinners) are in `assets/main.css`. SweetAlert2 is themed with custom CSS class overrides (`.vw-popup`, `.vw-btn-confirm`, etc.).

Token	Value	Usage
--color-primary	#6366f1 (indigo-500)	Buttons, links, active states
--color-success	#16a34a (green-600)	Positive amounts, confirmed status
--color-danger	#dc2626 (red-600)	Debit amounts, errors, delete actions
--color-warning	#d97706 (amber-600)	Pending status, unverified badge
--surface-main	#ffffff	Card / modal backgrounds
--surface-soft	#f8f9fc	Page background
--radius-md	10px	Cards, modals
--radius-xl	20px	Hero cards, large panels

8. Deployment

Docker Compose Services

Service	Image	Port	Description
virtual-wallet-db	mariadb:11	3306	MariaDB database. Root password: root. DB name: virtual_wallet.
virtual-wallet-backend	Dockerfile (eclipse-temurin:17-jre-jammy)	8080	Spring Boot application. Depends on db health check. JAR: backend-0.0.1-SNAPSHOT.jar

Build & Run

```
# Step 1: Build the Spring Boot JAR
cd backend
./gradlew build

# Step 2: Build Docker images and start all services
docker compose up --build

# Backend will be available at http://localhost:8080/api/v0
# Database at localhost:3306 (virtual_wallet)
```

Environment Variables

The docker-compose.yml passes environment variables that override application.properties values at runtime. Secrets are never committed to source control; they are loaded from application-secrets.properties locally or injected as env vars in CI/CD.

Variable	Default / Source	Description
SPRING_DATASOURCE_URL		DB connection URL (service name 'db' inside Compose)
	root	Database username
SPRING_DATASOURCE_USERNAME	root	Database password
JWT_SECRET	256-char hex string in properties	HMAC-SHA256 signing key
DEXCHANGE_RATE_API_KEY	application-secrets.properties	ExchangeRate-API v6 key
SPRING_MAIL_USERNAME	application-secrets.properties	Gmail SMTP username
SPRING_MAIL_PASSWORD	application-secrets.properties	Gmail App Password

Frontend Production Build

```
cd frontend
npm run build
# Static files in frontend/dist/
# Set VITE_API_BASE_URL env var before build to point at production backend
```

Health Check

The MariaDB service exposes a Docker health check that runs `mariadb-admin ping` every 5 seconds with a 10-retry limit. The Spring Boot service is configured with `depends_on: db: condition: service_healthy`, so the backend will not start until the database is accepting connections. This prevents connection-refused errors on fresh container startup.

Key Configuration Properties

```
# src/main/resources/application.properties
spring.datasource.url=jdbc:mariadb://localhost:3306/virtual_wallet
spring.datasource.driver-class-name=org.mariadb.jdbc.Driver
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jackson.property-naming-strategy=SNAKE_CASE
jwt.secret=<256-char hex key>

# src/main/resources/application-secrets.properties (NOT committed)
exchange.rate.api.key=YOUR_KEY
spring.mail.username=your@gmail.com
spring.mail.password=your_app_password
```